ECOTRACK TEAM EXECUTION PLAN

Member 2: Organization Application and GN Boundaries

A detailed database, PostGIS, backend, web, testing, migration, and handoff plan for citizen organization applications and official Grama Niladhari Division service-area selection.

Primary owner	Team Member 2
Primary branch	feature/organization-application-boundaries
Base branch	integration/organization-onboarding
Schema owner	Member 2 only for this milestone
Prepared	10 August 2026
Status	Implementation-ready

Contents

1. Mission and scope

Member 2 owns the citizen-facing organization application flow and the database structures/reference data that allow applicants to select official Sri Lankan Grama Niladhari (GN) Divisions as organization service areas. Member 2 is also the only person who changes `backend/prisma/schema.prisma` and creates migrations during this milestone.

> **Core outcome:** an authenticated citizen can submit valid organization details and one or more GN Division IDs. The backend creates a `PENDING_REVIEW` organization and pending service-area links. It does not approve the organization and does not create an ORG_ADMIN membership.

Included

- Reusable administrative-area table with GN names, codes, hierarchy, source metadata, and PostGIS boundary.
- Organization-to-administrative-area service links.
- Official GN reference-data import and validation.
- Prisma schema and migrations needed by Members 2 and 3.
- Citizen organization application endpoint and own-status endpoint.
- Citizen web application/status pages in the EcoTrack mockup style.

Explicitly excluded

- Super Admin approve/decline implementation (Member 3).
- First ORG_ADMIN membership creation (Member 3).
- Incident latitude/longitude submission.
- Point-to-GN lookup and organization incident visibility.
- Maps and incident dashboards.

2. Mandatory sources and collaboration rules

Read before schema design or importing spatial data:

- `docs/IMPORTANT/EcoTrack_Sri_Lanka_Service_Area_Boundaries_Guide.md`
- `docs/IMPORTANT/EcoTrack_Backend_Module_Structure_and_Heavy_Service_Reminder.txt`
- `docs/IMPORTANT/EcoTrack_Complete_System_Flow_Source_of_Truth_v1.txt`
- `database_docs/EcoTrack_Database_v2_Finalization_and_Codex_Instructions.txt`
- `database_docs/EcoTrack_ERD_v2_Final.dbml`
- `backend/prisma/schema.prisma` and existing applied migrations
- `docs/team-plans/VERY_IMPORTANT_TEAM_COLLABORATION_RULES.txt`

> **Never:** delete/recreate existing organization tables; edit an already-applied migration; commit raw credentials; invent GN boundaries without recorded source metadata; or create the requester's ORG_ADMIN membership during submission.

3. Branch and owned files

```
git fetch origin
git switch integration/organization-onboarding
git pull --ff-only origin integration/organization-onboarding
git switch -c feature/organization-application-boundaries
```

Owned areas

```
backend/prisma/schema.prisma
backend/prisma/migrations/<new-milestone-migrations>/**
backend/src/modules/administrativeAreas/**
backend/src/modules/organizations/application/**
backend/src/scripts/importAdministrativeAreas.ts (agreed location)
web/src/features/organizations/application/**
```

Do not directly change

```
backend/src/app.ts
backend/package.json (request dependency changes from Member 1)
web/src/App.tsx
backend/src/authorization/**
backend/src/modules/organizations/review/**
mobile/**
```

For local testing, mount the feature router inside an integration-test Express app. If temporary production wiring is necessary, keep it in a separate branch-only commit for Member 1 to review.

4. Current schema baseline

The schema already provides these models and they must be reused:

Existing model	Purpose in this milestone
UserProfile	Trusted requester profile; requester ID comes from authenticated request context.
Organization	Stores application details, requester, slug, review status and reviewer fields.
OrganizationVerificationDocument	Stores verification-document metadata if upload is included.
OrganizationMembership	Used later by Member 3 for FIRST_ADMIN; not created during submission.

Existing model	Purpose in this milestone
OrganizationMembershipRequest	For joining an existing organization; not the organization-creation request.
AuditLog	Submission may log an application-created event; review logs are Member 3.

The existing `Organization` defaults to `PENDING_REVIEW` and already records `requestedByUserId`. Preserve its unique slug and requester/reviewer relations.

5. Administrative-area reference design

The team selected the scalable reference-table option from the boundary guide: store each official GN Division once, then link organizations to it by ID.

Recommended conceptual model

Field	Requirement	Reason
id	UUID primary key	Stable application identifier.
level	Enum/value identifying GN	Allows future province/district/DS levels without ambiguity.
officialCode	Required, stable, unique with level/source	Idempotent imports and authoritative identity.
nameEn	Required	Current web display/search.
nameSi / nameTa	Nullable initially, preferred	Sri Lankan multilingual support.
districtName/code	Required when source provides it	Filtering and disambiguation.
divisionalSecretariatName/code	Required when available	GN hierarchy and user selection.
boundary	PostGIS geography/geometry MultiPolygon, SRID 4326	Official spatial representation and future lookup.
sourceName	Required	Provenance.
sourceVersion/date	Required	Boundary changes must be auditable.
sourceUrl / license	Record when available	Legal and verification trace.
isActive	Default true	Retire references without deleting history.
importedAt / updatedAt	Timestamptz	Operational trace.

Required constraints and indexes

- Unique stable area identity, such as `UNIQUE(level, official_code, source_version)` or an agreed source-independent official code constraint.
- GiST index on `boundary`.

- Normal indexes for district/DS/name selection filters.
- Boundary must be valid and SRID 4326.
- Do not rely on name alone as a unique key.

> **Schema decision required before migration:** choose whether a source-version update creates a new reference row or updates boundary/version metadata while retaining an import history. Do not silently replace geometry without an auditable decision.

6. Organization service-area design

This table links one organization to one selected GN reference. It does not duplicate the polygon.

Field	Requirement
id	UUID primary key
organizationId	FK to Organization
administrativeAreaId	FK to AdministrativeArea
status	PENDING_REVIEW / ACTIVE / DECLINED / SUSPENDED (agree exact enum)
reviewedByUserId	Nullable FK to UserProfile
reviewedAt	Nullable timestamptz
reviewNotes	Nullable
createdAt / updatedAt	Required timestamps

Required constraint:

```
UNIQUE (organization_id, administrative_area_id)
```

Submission creates links in the pending state. Member 3 activates or declines them in the organization review transaction.

7. Notification schema coordination

Member 3 owns notification behavior, but Member 2 owns the milestone schema. Agree on a minimal generic notification model before creating the migration:

Field	Purpose
id	UUID
recipientUserId	Authenticated EcoTrack UserProfile recipient

Field	Purpose
type	Stable event type such as ORGANIZATION_APPROVED / ORGANIZATION_DECLINED
title	Short UI title
message	Safe user-facing content
entityType / entityId	Optional navigation/reference to organization
readAt	Nullable timestamp
createdAt	Required timestamp

Add indexes for recipient plus read/created state. Do not store access tokens or secrets in notification metadata.

8. GN boundary acquisition and import

Source requirements

1. Prefer official Sri Lankan NSDI/Survey Department or another officially documented administrative dataset.
2. Record source URL, publisher, date/version, license, and original coordinate reference system.
3. Verify exact GN names and codes against the authoritative source.
4. Keep the raw source outside normal runtime code if licensing/size requires it, but document reproducible acquisition.
5. Never label community boundaries as legally official without verification.

Import pipeline

```
Acquire source file
  -> record checksum and metadata
  -> inspect CRS and attributes
  -> transform to EPSG:4326
  -> repair/validate geometry where justified
  -> normalize Polygon to MultiPolygon
  -> validate GN code/name/hierarchy
  -> idempotent upsert by official key
  -> verify counts, nulls, duplicates, SRID and validity
  -> build/verify GiST index
```

Import safety

- Import must be rerunnable without duplicate areas.
- Use a transaction or staged import where practical.
- Reject rows without stable codes or usable geometry into an error report.
- Log inserted, updated, skipped and failed counts.

- Do not perform a destructive full replacement against a shared database without explicit team approval.
- Test with a small known GN subset before importing the full dataset.

9. Migration workflow and validation

1. Update `schema.prisma` with agreed models/enums/relations.
2. Create a clearly named new migration.
3. Inspect SQL, especially PostGIS column type and GiST index.
4. Apply to the intended development database.
5. Generate Prisma client.
6. Run validation/status/database checks.
7. Commit schema, migration, generated client policy output, import code, and documentation together as agreed.

```
cd backend
npx prisma validate
npx prisma migrate dev --name add_administrative_areas_and_service_areas
npx prisma generate
npx prisma migrate status
npm run db:check
npm run typecheck
npm test
npm run build
```

> **Shared database caution:** confirm the target database and migration plan with the team before running `migrate dev`. Never reset the shared Supabase database and never edit an already-applied migration.

10. Organization application API

Proposed endpoints

Endpoint	Purpose
GET /api/v1/administrative-areas	Search/filter active GN references for selection.
POST /api/v1/organization-applications	Create pending organization and pending service-area links.
GET /api/v1/organization-applications/me	List/read applications submitted by authenticated user.
GET /api/v1/organization-applications/me/:id	Read own application details/status if needed.

Submission input

```
{
  "name": "Example Environmental Society",
  "registrationNumber": "...",
  "description": "...",
  "officialEmail": "...",
  "officialPhone": "...",
  "officialAddress": "...",
  "administrativeAreaIds": ["uuid", "uuid"]
}
```

Do not accept these from the body:

```
requestedByUserId
platformRole
organization status
reviewedByUserId
reviewedAt
membership role
membership status
```

Submission service transaction

```
Authenticated active USER
  -> validate DTO with Zod
  -> normalize names/email/phone as agreed
  -> validate selected area IDs exist and are active GN references
  -> generate/check slug
  -> transaction:
       create Organization(PENDING_REVIEW, requestedBy = auth profile)
       create pending OrganizationServiceArea rows
       create application-submitted AuditLog if required
  -> return safe application DTO
```

Do not create an OrganizationMembership. Member 3 creates the FIRST_ADMIN only after approval.

Business cases to decide/document

- Minimum/maximum number of GN areas per application.
- Whether one requester may have multiple simultaneous pending organization applications.
- Uniqueness/normalization policy for registration number and official email.
- Slug collision policy.
- Whether verification documents are required for this semester milestone.
- Whether applicants can edit pending applications; do not implement silently.

11. Backend module design

```
backend/src/modules/organizations/application/
  application.types.ts
  application.validation.ts
  application.dependencies.ts
  application.routes.ts
  controllers/
    createOrganizationApplication.controller.ts
    listMyOrganizationApplications.controller.ts
  services/
    createOrganizationApplication.service.ts
    listMyOrganizationApplications.service.ts
  repositories/
    organizationApplication.repository.ts
  application.integration.test.ts

backend/src/modules/administrativeAreas/
  administrativeArea.routes.ts
  administrativeArea.controller.ts
  administrativeArea.service.ts
  administrativeArea.repository.ts
  administrativeArea.types.ts
```

Responsibility boundaries

Layer	Responsibility
Route	Defines HTTP method/path and middleware order.
Controller	Reads validated request context/body, calls service, returns status/DTO.
Service	Business rules: selection validation, pending state, duplicates, slug policy.
Repository	Prisma queries and transactions only.
Validation	Zod schemas and normalized DTOs.

12. Citizen web pages

Use `ecotrack-srs-mockup` for the green EcoTrack design system, layout language, cards, fields, badges and responsive behavior. Do not copy mock-only logic.

Pages/components

- Organization application form
- GN Division search/filter and multi-select
- Selected service-area summary
- Submission confirmation
- My organization applications/status page

- Pending/declined/active status badges and review-note display where safe

Frontend rules

- Use the existing Supabase session and bearer `apiClient.ts`.
- Never send requester ID, approval status, reviewer ID or role.
- Disable double submission while request is running.
- Display backend validation errors safely.
- Use accessible labels, keyboard-operable multi-selection and responsive layout.
- Export pages for Member 1; do not replace production `App.tsx`.

13. Security and authorization contract

Member 1 provides CASL actions/subjects. Member 2's route must use those contracts. The backend requester is always:

```
request.authentication.profile.id
```

Own-application reads must include requester scope in the repository query. Searching active GN references may be available to authenticated users, but modification/import endpoints must not be public.

> **No tenant middleware on creation:** a citizen creating a new organization has no membership in that organization yet. Use authentication plus Create OrganizationApplication ability. Tenant middleware becomes relevant after an approved membership exists.

14. Testing strategy

Schema/import tests

- Schema validates and migration applies cleanly.
- GN official identity key is unique.
- Import rerun creates no duplicates.
- All imported geometry uses SRID 4326 and expected MultiPolygon type.
- Invalid/null geometry is reported, not silently inserted.
- GiST index exists.
- District/DS/GN counts and known samples are verified.

Application tests

- Missing/invalid bearer token rejected.
- Valid user can create pending application.
- Requester ID in body is ignored/rejected.
- Unknown/inactive area ID rejected.

- Duplicate area IDs rejected or normalized deterministically.
- Empty or excessive selection rejected by agreed policy.
- Slug collision handled safely.
- Transaction rollback leaves no orphan organization/service-area rows.
- No membership is created on submission.
- User can read only their own applications.

Test without production app.ts

```
const app = express();
app.use(express.json());
app.use("/api/v1", createOrganizationApplicationRouter(testDependencies));
app.use(errorMiddleware);
```

Run directly:

```
npx tsx --test src/modules/organizations/application/application.integration.test.ts
```

15. Handoff to Member 3

Member 3 must not write the real Prisma review repository until this checklist is complete:

- ☐ Schema models/enums reviewed by Member 1 and Member 3.
- ☐ Migration committed and applied successfully.
- ☐ Prisma client generated.
- ☐ Administrative-area import documented and sample/full import completed as agreed.
- ☐ Organization application creates PENDING_REVIEW organization.
- ☐ Selected service areas are pending and linked by IDs.
- ☐ Notification model required by Member 3 is present.
- ☐ Exact status enums and transaction expectations documented.
- ☐ Member 3 receives commands to update branch and verify migration status.

Member 3 may then use, but not alter

```
prisma.organization
prisma.organizationServiceArea
prisma.organizationMembership
prisma.auditLog
prisma.notification
```

16. Definition of done

- ☐ Correct feature branch and no unrelated edits.
- ☐ Existing organization models reused.
- ☐ Administrative-area and service-area designs documented and migrated.
- ☐ Official GN source/version/license metadata preserved.
- ☐ Import is idempotent and spatial validations pass.
- ☐ Citizen application creates only pending records.
- ☐ No ORG_ADMIN membership is created before approval.
- ☐ Own-application access is scoped by authenticated profile.
- ☐ Citizen web pages use real API and EcoTrack style.
- ☐ Member 1 receives exported router/page contracts.
- ☐ Member 3 receives the completed migration handoff.
- ☐ Prisma validate/status, database check, tests, typecheck, build and web lint pass.
- ☐ No incident/map work or secrets included.

Reference material

- Local source: `docs/IMPORTANT/EcoTrack_Sri_Lanka_Service_Area_Boundaries_Guide.md`
- Local source: `database_docs/EcoTrack_Database_v2_Finalization_and_Codex_Instructions.txt`
- Local source: `database_docs/EcoTrack_ERD_v2_Final.dbml`
- Local collaboration guide: `docs/team-plans/VERY_IMPORTANT_TEAM_COLLABORATION_RULES.txt`
